OWNAI / RESEARCH SERIES

Technical Architecture

Trust boundaries, local runtime, cloud relay, data models and API contracts

03

DETAILED TECHNICAL DOSSIER

User-owned accounts / Local-first runtime / Optional cloud relay

Research cutoff: 26 October 2026

A Consumer-web adapter (permission-gated)

B Official account authorization where supported

C User-owned free APIs and locally running models

INSIDE THIS VOLUME

01 Architectural decision and system boundaries

02 Component responsibilities

03 Local request path

04 Remote request sequence and connection lifecycle

05 API facade: smallest useful compatibility contract

06 Router and permissions algorithm

07 Data models and retention proposal

08 Operational modes and failure behavior

09 Reliability, performance and observability

10 Architectural decision records

11 Architecture acceptance checklist

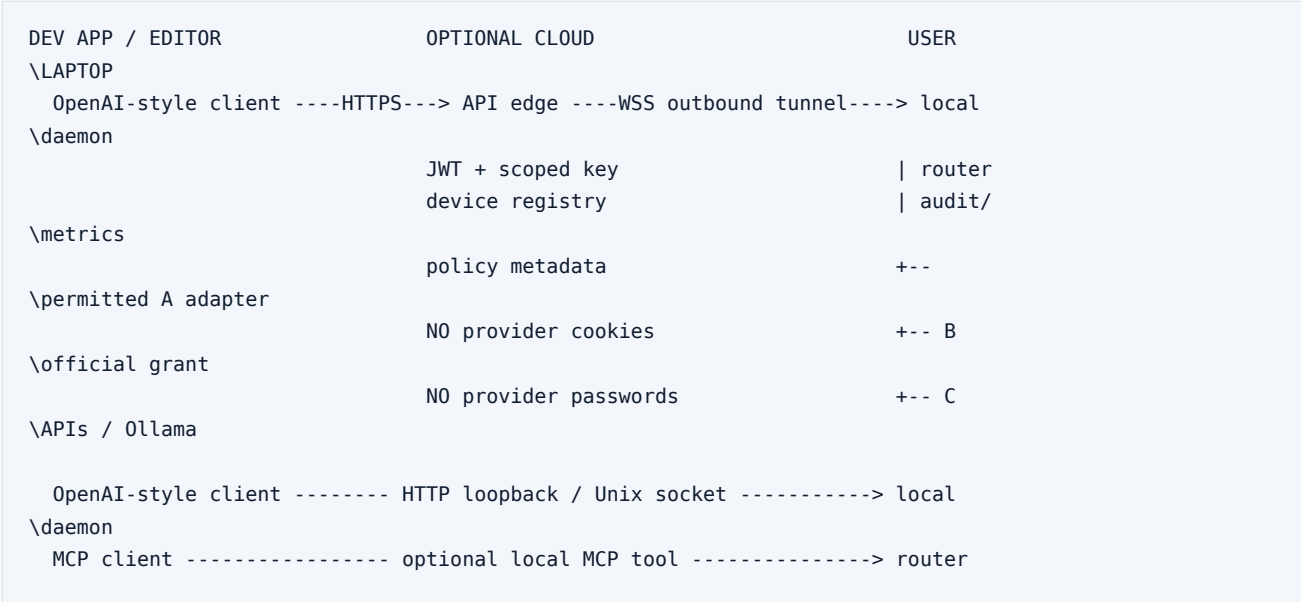
> Research notation: [Sxx] markers link to first-party terms, official technical documentation, published standards or original project repositories. Statements labeled proposed, target, or estimate are design judgments, not observed performance.

Architectural decision and system boundaries

Recommendation: an API-first, local-execution application with an optional online control plane and remote relay. Every account belongs to one person. The bridge runs on that person's device, owns the provider-specific login context and routes only to their authorized connections. The dashboard never stores provider passwords, browser cookies or account-session tokens. This is a design constraint, not proof that any particular consumer-chat connector is permissible. **[S01] [S02] [S03]**

The architecture has three distinct functions: **control plane** (accounts, device enrollment, policy), **data plane** (requests and responses), and **connector plane** (per-provider official API, authorized account grant, or approved browser integration). The Model Context Protocol is an optional consumer-facing tool layer above the same gateway; its evolving sampling semantics are not a substitute for a conventional generation API. **[S14]**

Trust-zone map



Remote operation necessarily places request content in transit through the cloud relay unless the system adds audited end-to-end payload encryption; HTTPS alone encrypts each leg, not against the relay itself. Show this prominently in the UI. Users who need local-only confidentiality must use the loopback endpoint.

Component responsibilities

Component	Responsibility	Trust or failure property
Desktop bridge	Bind loopback listener; authenticate requests; execute router; host encrypted local store	Requests stop when process exits
Adapter runtime	Manage provider session or official authorization; translate requests; normalize outputs	Adapter is permission-gated and capability-limited
Router	Choose explicit account/model using user policy, feature checks, quota and cost ceiling	Never silently change data recipient or charge
Local SQLite	Device settings, connection references, request statistics and non-sensitive state	Stored in OS user profile; encrypted sensitive fields
OS keychain	Protect provider tokens, API secrets and device signing key	No credential export to dashboard
Cloud API	Login, device registry, API key issuance, remote authorization, billing for hosted management if any	Stores only the minimum metadata
Cloud Postgres	Device and owner IDs, hashed API keys, audit events, consent flags	Row-level authorization and deletion
Relay	Multiplex authenticated, bounded request streams over user-initiated connection	Stateless where possible; no payload persistence
Observability	Aggregated request count, class of error, latency, device uptime	No raw prompt/response logging by default

Reference stack: TypeScript/Node.js + Fastify local bridge; Playwright only for approved browser connectors; SQLite local; desktop wrapper via Tauri or an installer with native process management; Next.js dashboard; Postgres hosted database; outbound TLS WebSocket tunnel; optional Redis for ephemeral presence/rate budgets. This stack is a proposal, not a provider requirement. Playwright supports persistent signed-in browser states but warns authentication files contain secrets; browser profile theft must be in the threat model. **[S21] [S23]**

Local request path

- The client issues `POST http://127.0.0.1:PORT/v1/chat/completions` using a local gateway key and an explicitly advertised model alias.
- Request parser validates JSON, max body size, allowed modalities and a short client deadline. The same auth is required on loopback; protect against cross-origin requests and SSRF into localhost.
- Router loads the user's active policy, resolves `model` to one concrete connection, checks adapter capabilities, quota/backoff, feature requirements and paid-cost restrictions.
- Adapter translates the normalized message list into provider-specific request shape. For A, compatibility is at best an approximation: chat UIs may not expose deterministic sampling parameters, exact model IDs, tool protocols or reproducible system-role semantics.

- Provider returns text/chunks; adapter records verified source and uncertainty fields, then normalizes result or error.
- Response includes a generated request ID, connection category and truthful model-confidence metadata. Input/output token totals are omitted or labeled estimates unless measurable.

Local requirement: bind `127.0.0.1` and `::1` only; reject default `0.0.0.0`. Prefer an OS socket for privileged applications. Authentication keys are scoped and revocable. Protect unauthenticated preflight behavior and validate Origin/Host; a web page visiting a user's browser must not gain arbitrary local bridge access. **[S19] [S37]**

Remote request sequence and connection lifecycle

```
Client -> Cloud API: POST /v1/chat/completions + remote key
Cloud -> Database: authenticate owner / device / scope / quota
Cloud -> Presence: find live enrolled session
Cloud -> Local daemon: WSS frame request ID + expiry + payload
Local daemon -> Local router: verify device scope and consent
Local router -> Provider: approved account-specific inference
Provider -> Local daemon: result or mapped error
Local daemon -> Cloud: bounded response frames
Cloud -> Client: HTTP response or SSE stream
```

Enrollment: the user logs into the cloud and generates a one-time pairing code. The daemon creates a non-exportable or OS-keychain-protected device key and proves possession against a short-lived enrollment challenge. The server binds device ID to the authenticated user, and the daemon displays the account and remote-consent scope before opening the tunnel. Re-enrollment invalidates old keys. Session recovery has exponential backoff and jitter; device offlining immediately rejects remote new work.

Each WSS message has `{version, request_id, device_id, expires_at, message_type, body}` and an authenticated session binding. The relay must validate the target device owner for **every** request; a connected socket does not authorize arbitrary operations. Limit frame sizes, pending requests and byte volume; use TLS/WSS, ping/pong heartbeat, replay-resistant request IDs and cancellation propagation. **[S19] [S28] [S38]**

API facade: smallest useful compatibility contract

```
GET  /health
GET  /v1/models
POST /v1/chat/completions
POST /v1/responses                # later, only as supported
GET  /v1/bridge/capabilities      # extension
GET  /v1/bridge/connections       # local admin, separate scope
```

```
{
  "model": "auto:free-text",
  "messages": [{"role": "user", "content": "Explain closures simply"}],
  "stream": false,
  "max_tokens": 256
}
```

On success, return standard-shaped `choices[0].message.content` for text-only workloads, a unique `id`, `created`, `model`, and a `bridge` extension containing `provider`, `connection_kind`, `model_confidence`, `local_or_remote`, `quota_status`. Do not report fabricated exact token consumption. Treat unsupported fields as explicit `unsupported_parameter` or `unsupported_feature`, not silent success. Browser adapters cannot promise genuine OpenAI tool-call behavior.

For SSE streaming, emit complete ordered `chat.completion.chunk` objects and end with `[DONE]`; if an adapter only yields the final page answer, either buffer and send one final chunk (labeled not realtime) or reject requests that explicitly demand low-latency token streaming. Handle client disconnect by canceling inflight local work where provider support allows. Never retry a non-idempotent generation automatically after uncertain completion.

Router and permissions algorithm

```
input: requested alias, modalities, privacy mode, maximum spend, fallback setting
candidate set = user's own enabled and authorized connections
filter candidate set by provider permission gate and proven capabilities
filter by available quota, timeout state, privacy rule and explicit cost cap
choose user's preferred candidate; persist request trace without content
if quota/rate failure, consider next ONLY if user explicitly approved its
\destination
if same compatibility level unavailable, return actionable normalized error
```

Fallback **changes the processor of user data**. Ask consent per source/destination class (e.g. Gemini free API to local Ollama), not merely per price class. Default is no fallback. If a user permits fallback, expose the actual selected provider and notify when a route changed. Rate limits should be enforced at gateway and adapted to provider 429s; provider quotas are often dynamic. **[S10] [S11]**

Data models and retention proposal

Store / object	Example fields	Secret? / retention proposal
Cloud `users`	opaque ID, sign-in method, preferences	No provider credentials; until account deletion
Cloud `devices`	ID, owner, public signing key, last seen, revoked	No session secret; until revoked + 30d log tombstone
Cloud `gateway_keys`	salted hash, prefix, scopes, owner, expires	Never store plaintext; rotate and purge on revoke
Cloud `request_events`	request ID, route class, status, ms, bytes band	No content; aggregate after 30d, delete raw after 7d by default
Local `connections`	provider ID, status, capability, encrypted secret reference	Device only, delete on disconnect
Local `routing_policies`	ordered permitted connections, fallback consent, budget	Device only unless user syncs sanitized policy
Local `request_events`	request ID, timestamp, error class, latency	7d default or disabled; no prompt/response bodies
Local browser profile	Provider-owned cookies/session state	Sensitive, device only, isolated directory and deletion control

Retention numbers are **design defaults requiring legal and user review**, not established regulatory minima. Avoid cloud model-level conversation history; the bridge need not become a chat archive.

Operational modes and failure behavior

Mode	Availability	Secret location	Typical use
Local-only	Device agent running	Local keychain / provider browser state	Private coding and tinkering
Remote paired	Device online plus cloud relay healthy	Provider credentials still local; gateway key hashed cloud	Calling from VPS or another phone/device
Free official API	Internet + account quota	User API credential stored local	Stable documented developer integration
Ollama	Compatible local hardware and service running	No proprietary provider account needed	Offline/private text generation **[S13]**

Failure policy: fail closed. Offline device yields HTTP 503 `device_offline` with a retry suggestion; cloud outage does not stop local-only calls; lost provider login yields `login_required`; mismatch of features yields 400; permission-disabled connector yields 403. Apply a bounded request queue (e.g. 10) and one concurrent browser UI session per profile initially. Browser operations should use isolated persistent profiles and foreground-inference cues where practical.

Reliability, performance and observability

- Instrument `request.accepted`, `routed`, `provider.started`, `first_byte`, `finished`, `failed`, `cancelled` with monotonic durations and correlation IDs.
- Dashboard reflects heartbeat age rather than inventing online status; the `last_seen` field should be displayed to users.
- Use two timeouts: provider-init and absolute request deadline (for example, 20s first-byte and 120s total, configurable); some browser routes need a slower documented profile.
- Set limits for response size, upload size (text-only v1), per-key concurrency and retry budgets.
- Update adapters behind version and health flags; remotely disable a broken/unauthorized provider connector without deleting user data.
- Publish an explicit status/limitations page. No SLA promise for third-party consumer UI access.

Architectural decision records

ADR	Decision	Rejected alternative / reason
ADR-001	Local credential custody	Cloud-captured cookies create unacceptable breach and provider-risk concentration
ADR-002	REST first, optional MCP wrapper	MCP is not a universal direct replacement for model APIs **[S14]**
ADR-003	Official provider route as reliable baseline	A browser bridge alone cannot offer stable permission and capability guarantees
ADR-004	Opt-in relay without inbound ports	Exposing local HTTP to the public internet multiplies home-network attack surface
ADR-005	Keyed per-connection routing	Opaque random fallback can leak data or consume unapproved paid credits
ADR-006	Preserve model uncertainty	Consumer UIs do not always disclose precise model provenance

Architecture acceptance checklist

- Local access always requires a valid key; no cross-origin webpage can reach generation endpoints unauthenticated.
- Device pairing proof, owner authorization and key revocation are tested end-to-end.
- Provider account cookies/tokens never appear in cloud database, network telemetry or diagnostic export.
- Remote 503 is reliable when device shuts down; local route still works if cloud is unreachable.
- A new provider adapter can be added without changing client SDK or dashboard tables.
- Every declared supported capability is verified with a repeatable test; simulated capabilities are not advertised.

Evidence & source register

Sources were checked in October 2026. Their content, access conditions, prices and quotas can change. Project READMEs are evidence of what authors claim to implement, not proof of provider permission or reliability.

[S01] OpenAI consumer Terms of Use (effective January 2026)
https://openai.com/policies/row-terms-of-use/

[S02] Anthropic Consumer Terms of Service (effective October 2025)
https://cdn.sanity.io/files/4zrzovbb/website/43c13f51a6d9755341433f79cf63c966ed9200ea.pdf

[S03] Anthropic Claude Code - Legal and compliance
https://code.claude.com/docs/en/legal-and-compliance

[S10] Google Gemini API rate limits
https://ai.google.dev/gemini-api/docs/rate-limits

[S11] GroqCloud API rate limits
https://console.groq.com/docs/rate-limits

[S13] Ollama OpenAI-compatible API
https://docs.ollama.com/api/openai-compatibility

[S14] MCP specification release, 28 July 2026
https://blog.modelcontextprotocol.io/posts/2026-07-28/

[S19] OWASP WebSocket Security Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/WebSocket_Security_Cheat_Sheet.html

[S21] Playwright authentication and browser-state guidance
https://playwright.dev/docs/auth

[S23] Tauri - Node.js sidecar packaging
https://v2.tauri.app/learn/sidecar-nodejs/

[S28] OWASP API Security Top Ten, 2023
https://api-security.owasp.org/editions/2023/en/0x11-t10/

[S37] OWASP Cross-Site Request Forgery guidance
https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

[S38] OWASP API Broken Object Level Authorization
https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/

> Evidence boundary: this series is a design-and-research package, not a working implementation, legal opinion, measured security certification or promise of free access to any specific proprietary model.